

Basic Action Statements in a Function or Procedure

1. Get the named values passed in as *input* parameters.
2. Reserve memory space for a *variable* of some *type* and give it a name. The name is used to manipulate the value(s) stored in the memory space. The value(s) within a variable may change throughout the run of the routine
3. *Assign* a value to go into a variable or into a field of an object. The value can be the result of some *operation* or can be *read as input* from a file location or the user console.
4. *Call* on some previously defined routine to manipulate variables or to perform a calculation.
5. *Return* the results of the calculation or computations

Operations: (Store the result into a variable of appropriate type)

- Arithmetic Operations: +, -, /, *, % (for numeric variables)
- Comparison Operations: <, >, = (for evaluating conditions on the variables)
- Logical Operations: AND, OR, NOT (for evaluating conditions on the variables)
- Character Operations (for text processing)
- Advanced operations involving Sets, Matrices, and other object types. (Must be defined)

Control Structures: (Dictate the order in which operations are performed)

- BLOCKS allow a sequence of program steps to be treated as a single unit; they can be nested.
- IF (a condition holds)
 - THEN perform some step or block of steps
 - ELSE perform a different step or block of steps
- LOOPS allows a block of steps to be repeated:
 - FOR LOOP: perform the block a pre-specified number of times.
 - WHILE LOOP: repeatedly perform the block as long as some condition continues to hold.
(Beware of Infinite Loops – the condition must become untrue at some time so loop can end.)

For Program Clarity:

- Comments, Remarks
- Program description, Input / Output Specifications, Variable definitions